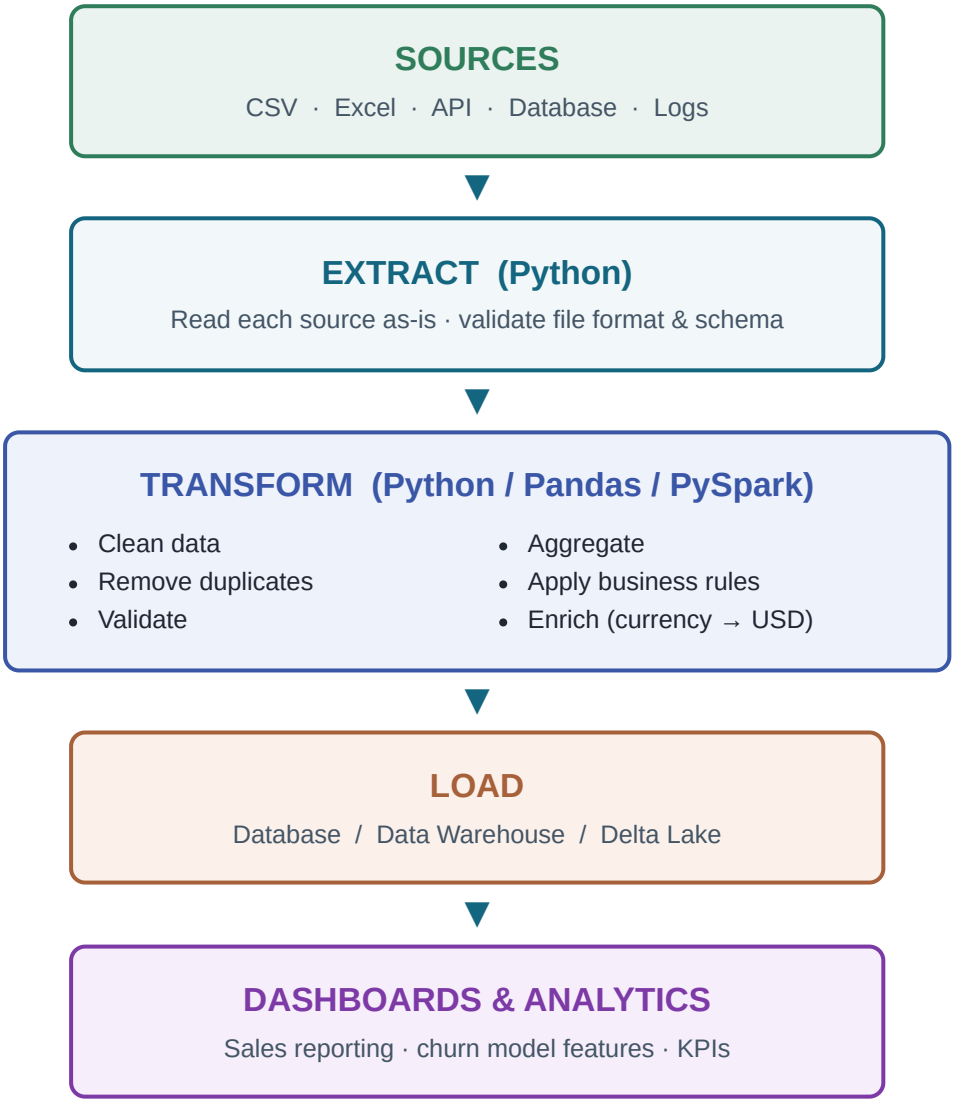
NovaCart ETL Pipeline

End-to-End Data Flow · AI-Engineering Exercise

The same NovaCart datasets flow through five stages. This document shows the pipeline structure end-to-end and consolidates **all transformation constraints into one table**.

Pipeline Flow



Sources — the NovaCart raw feeds

Source file	Type	Description
`orders.csv`	CSV	Core sales transactions (order line grain)
`customers.json`	API / JSON	Customer master (nested address)
`products.csv`	CSV (pipe “|”)	Product catalog with per-product currency price

`returns.tsv`	Excel / TSV	Returned orders (tab-separated)
`exchange_rates.json`	API	Currency → USD conversion rates
`web_events.log`	Logs	Clickstream events (view / cart / checkout / purchase)

Transform Constraints — single consolidated table

Every rule the Transform stage must enforce, across all datasets. Rejected rows are written to a `reject_log` with a reason — never silently dropped.

#	Category	Field(s)	Constraint / Rule	On violation
1	Clean	`order_id` & all string fields	Trim leading/trailing whitespace; standardise case	Auto-fix
2	Deduplicate	`order_id`	Must be unique after trimming (drop duplicates, keep first)	Drop duplicate
3	Validate	`transaction/order_id, product_id, sale/order_date`	Must not be null	Reject row
4	Validate	`customer_id`	Must be present and exist in customers	Reject row
5	Validate	`quantity`	Must be > 0 (no zero / negative)	Reject row
6	Validate	`price / amount`	Must be numeric and ≥ 0 (not blank)	Reject row
7	Type cast	`sale_date / order_date`	Parse to a real date from any of 3 formats (YYYY-MM-DD, DD/MM/YYYY, MM-DD-YYYY)	Reject if unparseable
8	Standardise	`country`	Map variants to canonical (USA / India / UK / Germany)	Auto-fix
9	Flag	`email`	Add `email_present` boolean	Derive
10	Enrich	`currency, price`	Join `exchange_rates`; `amount_usd = amount × rate_to_usd`	Reject if currency unknown
11	Business rule	`quantity, price, discount`	`total = qty×price`; `discount_value = total×discount/100`; `net_revenue = total − discount_value`	Derive
12	Classify	`price`	`price_tier`: low <10, medium 10–<100, high ≥100	Derive
13	Referential	`product_id`	Must exist in products catalog	Reject row
14	Enrich	`order_id` vs returns	Left-join returns → add `is_returned` flag (keep non-returned)	Derive

15	Aggregate	per `customer_id`	total_orders, total_spend_usd, avg_order_value, return_rate, days_since_last_order	Derive (feature table)
16	Aggregate	per `customer_id` (web_events)	sessions_count, cart_abandon_rate	Derive (feature table)
17	Audit	all rejected rows	Write to `reject_log` with `reject_reason`	Always log

Re-runnability: the Transform stage must be deterministic — running it twice on the same input produces identical output, with no duplicates.

Load & Analytics targets

Layer	Output	Purpose
Load	`fact_sales` (Delta / SQL)	Clean order lines, USD, returned flag
Load	`dim_customers`, `dim_products`	Standardised dimensions for slicing
Load	`feature_customers`	ML-ready churn features (one row/customer)
Analytics	Dashboards	Revenue by tier / country / period; return rate; churn